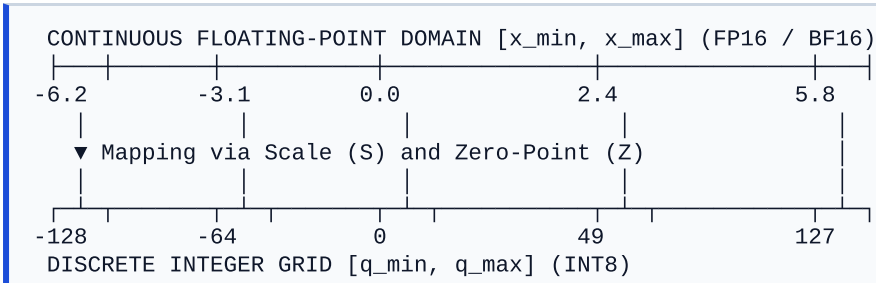
TECHNICAL REFERENCE & CURRICULUM GUIDE

Module 11 (Part 1): Quantization & Inference Optimization

Module 11 Part 1 covers the numerical foundations, theoretical algorithms, hardware execution dynamics, and practical implementations of post-training weight quantization for Large Language Models—with a rigorous focus on second-order Hessian optimization and the GPTQ algorithm.

11.1 Quantization Fundamentals: INT8/INT4, Symmetric vs. Asymmetric Quantization

Quantization projects high-precision continuous floating-point tensors (typically 16-bit Brain Floating Point BF16 or standard FP16) onto lower-bit discrete integer representations (such as INT8 or INT4). The primary objective is to shrink memory bandwidth requirements and accelerate linear algebra operations (GEMM) while preserving the model's representational capacity.



1. The Core Mapping Functions

Quantization maps a continuous real-valued tensor $X \in \mathbb{R}$ to an integer grid $Q \in [q_{min}, q_{max}]$ via two fundamental parameters:

1. **Scale Factor ($S \in \mathbb{R}^+$):** A positive real number determining the step size between adjacent discrete quantization levels.
2. **Zero-Point ($Z \in \mathbb{Z}$):** An integer offset ensuring that real-valued zero (0.0) maps exactly to a representable integer grid point. This is mathematically required to guarantee that zero-padding, dropout masks, and ReLU activations do not introduce systematic numerical bias.

Forward Quantization:

$$q = \text{clip}\left(\lfloor \frac{x}{S} \rceil + Z,\ q_{\min},\ q_{\max} \right)$$

where $\lfloor \cdot \rceil$ denotes the round-to-nearest-integer operator, and $\text{clip}(v, l, u) = \max(l, \min(v, u))$.

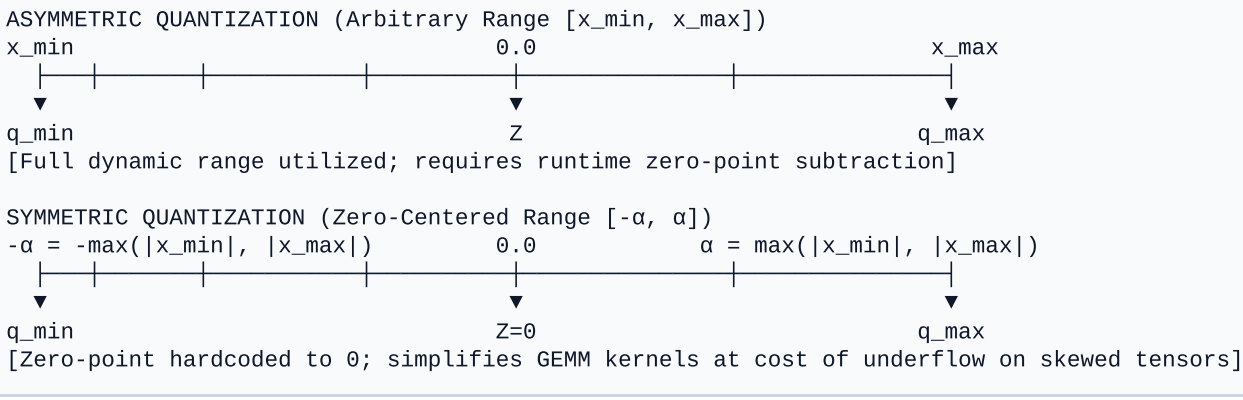
Dequantization:

$$\hat{x} = S \cdot (q - Z)$$

The quantization error for any scalar value is defined as:

$$\varepsilon = x - \hat{x} = x - S \cdot \left(\text{clip}\left(\left\lfloor \frac{x}{S} \right\rceil + Z,\ q_{\min},\ q_{\max} \right) - Z \right)$$

2. Symmetric vs. Asymmetric Quantization



Asymmetric Quantization (Affine Quantization)

Maps the arbitrary real-world dynamic range $[x_{min}, x_{max}]$ directly onto $[q_{min}, q_{max}]$:

$$S = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \quad Z = \text{round}\left(\frac{-x_{\min}}{S} \right) + q_{\min}$$

- **Advantage:** Maximizes resolution when the underlying distribution is skewed or strictly non-negative (e.g., activations after ReLU or GeLU where values are predominantly ≥ 0).
- **Disadvantage:** During matrix multiplication, the non-zero Z introduces an offset cross-term that requires runtime integer subtraction:

  $$Y = X \cdot W = [\, S_X (Q_X - Z_X) \,] \cdot [\, S_W (Q_W - Z_W) \,] = S_X S_W \left(Q_X Q_W - Z_W Q_X - Z_X Q_W + Z_X Z_W \right)$$

  The terms $Z_W Q_X$ and $Z_X Q_W$ add runtime overhead to GPU tensor cores unless fused or hoisted out of loops.

Symmetric Quantization (Scale-Only Quantization)

Restricts the zero-point to zero ($Z = 0$) by forcing the real-valued range to be strictly symmetric around the origin: $[-\alpha, \alpha]$, where $\alpha = \max(|x_{min}|, |x_{max}|)$.

$$S = \frac{\alpha}{q_{\max}}, \quad Z = 0$$

For signed b-bit integers (where $q_{min} = -2^{b-1}$ and $q_{max} = 2^{b-1} - 1$):

$$q = \text{clip}\left(\left\lfloor \frac{x}{S} \right\rceil, -2^{b-1}, 2^{b-1} - 1 \right), \quad \hat{x} = S \cdot q$$

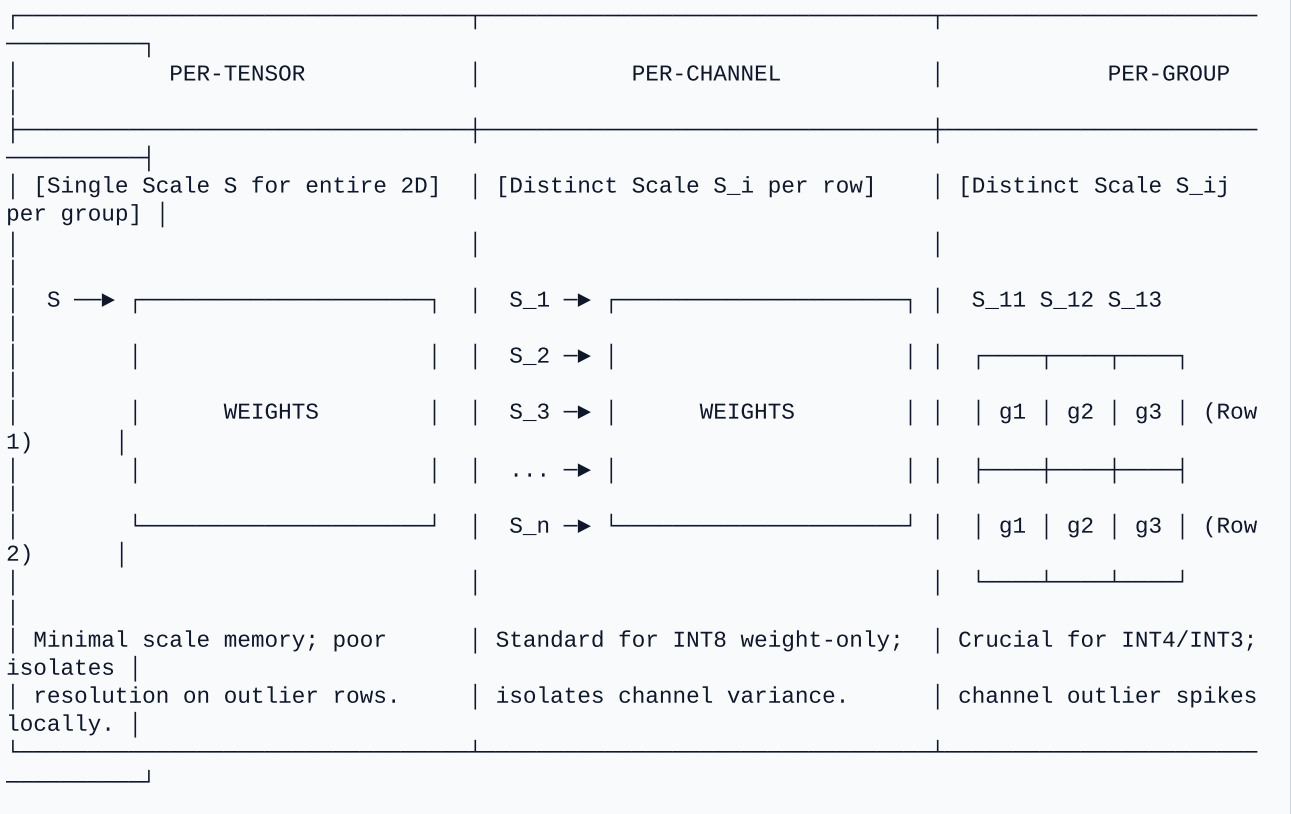
- **Advantage:** Eliminates the zero-point terms entirely from linear algebra execution:

  $$Y = X \cdot W = (S_X Q_X) \cdot (S_W Q_W) = (S_X S_W) \cdot (Q_X Q_W)$$

  Hardware tensor cores directly compute the integer dot-product matrix multiplication $Q_X Q_W$ natively via instructions like `DP4A` (on older architectures) or `MMA` (Matrix Multiply Accumulate on Ampere/Hopper/Blackwell).
- **Disadvantage:** If a tensor's values are strictly positive (*[0, 10]*), the symmetric range expands to *[−10, 10]*, wasting 50% of the discrete integer states on negative values that never occur, effectively discarding 1 bit of precision.

3. Quantization Granularities: Per-Tensor, Per-Channel, and Per-Group

The scale factor S can be shared across different structural granularities within a weight matrix $W \in \mathbb{R}^{out_features \times in_features}$.



1. **Per-Tensor:** A single scale S and zero-point Z govern the entire matrix. Minimal storage overhead, but high error: a single extreme outlier forces the scale factor S to expand, causing all small weights to collapse to 0.
2. **Per-Channel (Per-Row / Per-Output-Feature):** Each output channel (row of the weight matrix) receives its own scale S_i. This isolates rows with high variance from rows with low variance. Standard for INT8 weights.
3. **Per-Group (Sub-Channel Grouping):** Each row is subdivided into contiguous groups of size G (typically $G \in \{32, 64, 128\}$). A unique scale $S_{i,j}$ and zero-point $Z_{i,j}$ are allocated to every individual group. This is the **industry standard for 4-bit and 3-bit quantization (GPTQ, AWQ)** because it limits the blast radius of weight outliers to exactly G elements.

11.2 Post-Training Quantization (PTQ) vs. Quantization-Aware Training (QAT)

Compression can be applied either after pre-training completes (PTQ) or baked directly into the training loop (QAT).

```
POST-TRAINING QUANTIZATION (PTQ)
[Pre-trained Model (FP16)] ⟶ [Small Calibration Data (128-512 samples)] ⟶ [Static
Quantization Algorithm (GPTQ/AWQ)] ⟶ [Quantized Model (INT4)]
Compute Required: Minutes to a few hours on 1x GPU. No full backpropagation.

QUANTIZATION-AWARE TRAINING (QAT)
[Base Model (FP16)] ⟶ [Full Forward Pass] ⟶ [Fake-Quantization Nodes] ⟶ [STE
Backpropagation] ⟶ [Optimizer Step] ⟶ [Quantized Weights]
Compute Required: Days to weeks on multi-GPU clusters. Requires full pre-training/SFT data
pipeline.
```

1. Post-Training Quantization (PTQ)

PTQ takes an already trained FP16/BF16 checkpoint and converts the tensors directly to low-bit integers without performing standard iterative training.

- **Input Requirements:** A small calibration set (128–512 token sequences) to record dynamic activation ranges.
- **Algorithmic Spectrum:**
  - *Naive Round-To-Nearest (RTN):* Rounds values to the nearest integer grid point without taking activation magnitude into account. Fails catastrophically at 4 bits.
  - *Second-Order Optimization (GPTQ, AWQ):* Inverts local Hessian matrices or scales salient channels to minimize output activation error.
- **Strengths:** High operational throughput. An 8B model quantizes in under 5 minutes; a 70B model quantizes in ~1–2 hours on a single workstation.
- **Weaknesses:** At ultra-low bitwidths (≤ 3 bits) or on models smaller than 3B parameters, PTQ encounters an irreducible quantization noise floor, degrading benchmark perplexity.

2. Quantization-Aware Training (QAT)

QAT models quantization noise directly during pre-training or continued Supervised Fine-Tuning (SFT). The network weights are updated so that the converged loss landscape rests in flat minima that are resilient to discrete truncation.

The Straight-Through Estimator (STE)

The core mathematical challenge of QAT is that the derivative of the rounding function $\lfloor x \rceil$ with respect to x is zero everywhere and undefined at half-integers:

$$\frac{d \lfloor x \rceil}{dx} = 0 \quad (\text{for } x \notin \mathbb{Z} + 0.5)$$

If standard backpropagation is applied through a rounding operation, all gradients vanish ($\nabla_\theta L = \boldsymbol{0}$), halting parameter updates.

The **Straight-Through Estimator (Bengio et al., 2013)** resolves this by substituting the non-differentiable step function with an **identity pass-through during the backward pass**:

$$\text{Forward Pass: } \hat{w} = S \cdot \text{clip}\left(\left\lfloor \frac{w}{S} \right\rceil, q_{\min}, q_{\max} \right)$$

$$\text{Backward Pass: } \frac{\partial L}{\partial w} \approx \frac{\partial L}{\partial \hat{w}} \cdot \mathbb{I}\left(q_{\min} \leq \frac{w}{S} \leq q_{\max} \right)$$

where $\mathbb{I}(\cdot)$ is the indicator function acting as a saturation gate.

```
import torch

class StraightThroughEstimator(torch.autograd.Function):
    @staticmethod
    def forward(ctx, w: torch.Tensor, scale: torch.Tensor, q_min: int, q_max: int):
        # Save bounds for backward pass clipping
        ctx.save_for_backward(w, scale)
        ctx.q_min = q_min
        ctx.q_max = q_max

        # Real discrete rounding in forward pass
        scaled = w / scale
        clamped = torch.clamp(torch.round(scaled), q_min, q_max)
        return clamped * scale

    @staticmethod
    def backward(ctx, grad_output: torch.Tensor):
        w, scale = ctx.saved_tensors
        scaled = w / scale

        # STE: Pass gradient straight through, zeroing out values outside range
        mask = (scaled >= ctx.q_min) & (scaled <= ctx.q_max)
        grad_w = grad_output * mask.to(grad_output.dtype)

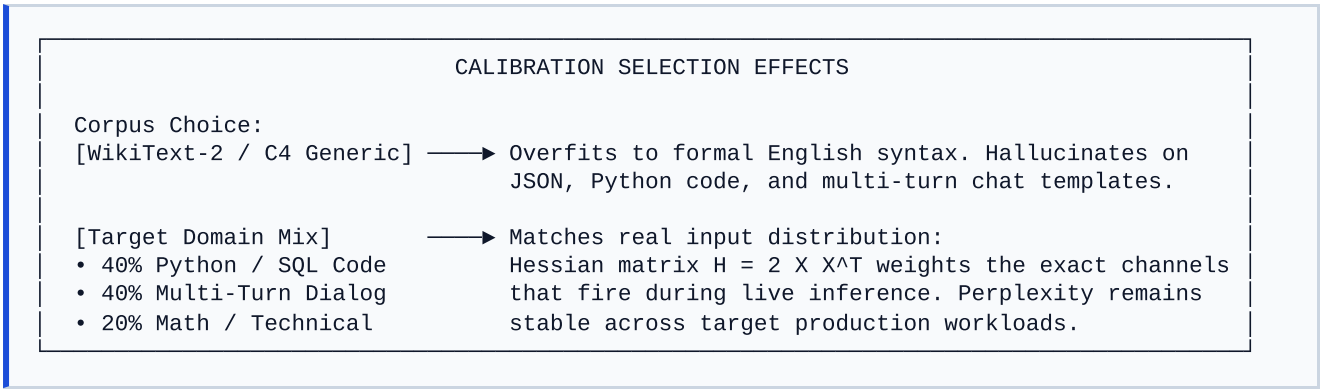
        return grad_w, None, None, None
```

PTQ vs. QAT Architectural Comparison

Dimension	Post-Training Quantization (PTQ)	Quantization-Aware Training (QAT)
Compute Cost	Negligible (~ 0.1–2.0 GPU hours)	High (~ 50–$1{,}000+$ GPU hours)
Dataset Requirement	128 to 512 unlabeled text sequences	Full domain/SFT corpus (10^7–10^{10} tokens)
Engineering Complexity	Simple CLI execution (`AutoGPTQ` , `llama.cpp`)	Complex training loops, STE tuning, stability monitoring
4-bit Accuracy	Matches FP16 on models ≥ 7B	Matches FP16 across all scales (including < 3B)
2-bit / 3-bit Accuracy	Unusable; severe perplexity divergence	Viable down to 2.5–3.0 bits

11.3 Calibration Datasets and Their Effect on Quantized-Model Quality

Advanced PTQ algorithms (like GPTQ and AWQ) do not look at weight tensors in isolation. They evaluate the **activation distribution** X elicited when the model processes representative language sequences.



1. The Role of the Calibration Set

During GPTQ calibration, the input activations X are passed through the frozen model to construct the empirical Hessian matrix:

$$H = 2\,X\,X^T$$

This matrix measures the curvature of the loss surface with respect to the layer's output activations.

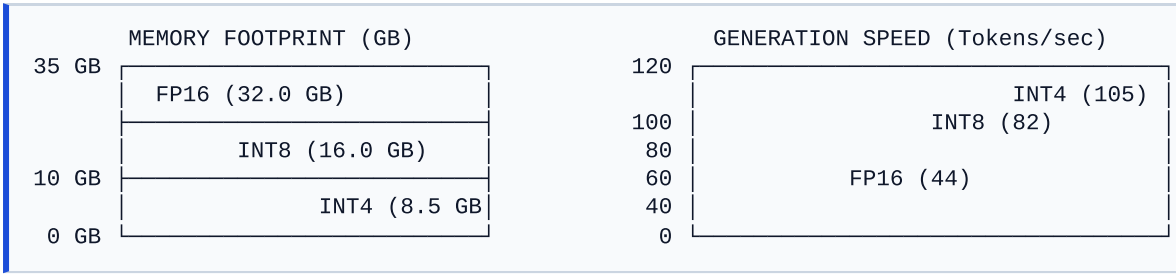
- Channels that fire frequently and with large amplitudes yield large eigenvalues in H.
- The algorithm uses these eigenvalues to assign **low rounding error tolerance** to high-salience weights, while pushing rounding noise onto weights corresponding to small eigenvalues.

2. Failure Modes: Domain Drift and Sequence Truncation

1. **Domain Mismatch / Specialization Collapse:** If a base model is calibrated exclusively on `WikiText-2` and deployed as a Python code copilot, the Hessian matrix underweights the channels responsible for tracking code indentation, brackets, and syntax trees. The model's HumanEval pass rate drops, despite showing good perplexity on Wikipedia text.
2. **Context Length Truncation:** Calibrating on short sequence lengths (e.g., 512 tokens) prevents the model from activating the RoPE (Rotary Position Embedding) features and long-range attention channels that fire at sequence lengths of 8k or 32k tokens. Calibration sequences must match the operational context length (minimum 2048 to 4096 tokens).
3. **Overfitting to Calibration Size:**
   - *Too few samples ($N < 64$):* The sample covariance matrix $(1 / N)\,X\,X^T$ is rank-deficient and noisy, causing the Cholesky decomposition of the Hessian to fail or produce unstable rounding trajectories.
   - *Too many samples ($N > 1024$):* Incurs diminishing returns while linearly scaling calibration wall-clock time and memory consumption.
   - *Production Standard:* **128 to 256 randomly shuffled sequences**, drawn from a blend of the target deployment distribution (e.g., ShareGPT conversation traces, technical documentation, and code).

11.4 Quality, Speed, and Memory Trade-Off Curves Across Quantization Levels

Quantization trade-offs are governed by hardware architecture: memory bus bandwidth vs. arithmetic tensor core throughput.



1. Memory Footprint and the Memory-Bound Regime

Large Language Model text generation operates in two distinct operational phases:

1. **Prefill (Prompt Processing):** Compute-bound. The entire input prompt is ingested in parallel via high-arithmetic-intensity matrix-matrix multiplications (GEMM).
2. **Decode (Autoregressive Generation): Memory-bandwidth bound.** The model processes one token at a time. For every single generated token, **100% of the model parameters must be read from High-Bandwidth Memory (HBM) into on-chip cache (SRAM)** to perform matrix-vector multiplications (GEMV).

The theoretical maximum decoding throughput (Tokens Per Second) is bounded by memory bandwidth:

$$\text{Max Tokens/sec} \approx \frac{\text{Hardware Memory Bandwidth (GB/s)}}{\text{Model Memory Footprint (GB)}}$$

Empirical Hardware Case Study: 70B Model on 1x NVIDIA A100 (80GB SXM4)

- **A100 Memory Bandwidth:** $\approx 2{,}039$ GB/s
- **FP16 Model Size:** $70 \times 10^9 \times 2$ bytes ≈ 140 GB (Cannot fit in a single 80GB GPU; requires a 2x A100 cluster minimum).
- **INT4 Model Size:** $70 \times 10^9 \times 0.5$ bytes + scales ≈ 38.5 GB (Fits into a single A100 with 41.5 GB remaining for KV cache).
- **Theoretical Maximum Single-Batch Throughput (INT4 on 1x A100):**

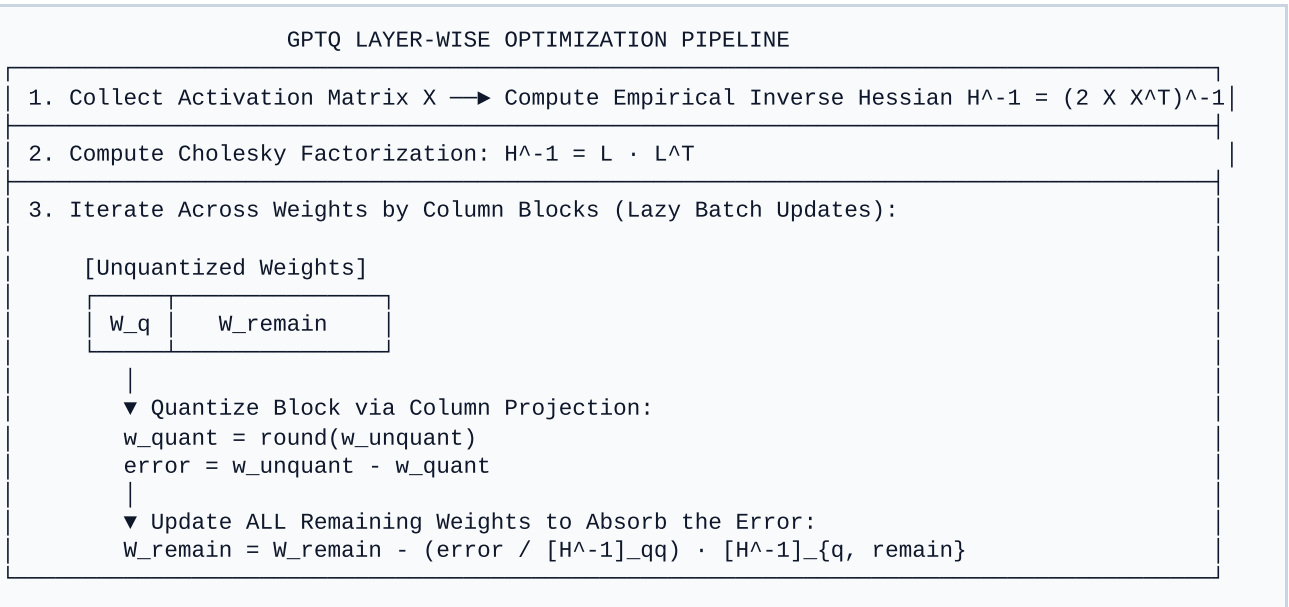
$$\text{Throughput} \approx \frac{2{,}039 \text{ GB/s}}{38.5 \text{ GB}} \approx \mathbf{52.9 \text{ tokens/sec}}$$

2. Comprehensive Quantization Benchmark Landscape (LLaMA-3-8B Baseline)

Format	Bits / Weight	Model Size	Zero-Shot Accuracy (% of FP16)	Decode Latency	Production Viability
FP16 / BF16	16	16.0 GB	100.0% (Baseline)	*1.0*× (Slowest)	Gold standard for training; inefficient for single-stream edge serving.
INT8 (Weight-Only)	8	8.5 GB	99.8%	*1.5*× – *1.8*× faster	Zero degradation across all model scales. Direct drop-in.
INT4 (GPTQ / AWQ, g=128)	4	4.8 GB	98.5% – 99.2%	***2.2*× – *3.2*× faster**	**Industry Sweet Spot.** Maximum compression before quality drop-off.
INT3 (Group Size 64)	3.25	3.9 GB	91.0% – 94.5%	*2.4*× – *3.3*× faster	Noticeable degradation on multi-step reasoning and coding tasks.
INT2 (Naive PTQ)	2	2.6 GB	*< 40.0%* (Collapse)	Unstable kernels	Models diverge; outputs degrade into repetitive token loops.

11.5 GPTQ Algorithm Deep Dive: Layer-Wise, Hessian-Based Weight Quantization

Developed by Frantar et al. (NeurIPS 2022), **GPTQ (Generalized Post-Training Quantization)** adapts Optimal Brain Surgeon (OBS) second-order Taylor series approximations to quantize models containing tens of billions of parameters in polynomial time.



1. The Mathematical Objective

Given a linear layer with pre-trained weight matrix $W \in \mathbb{R}^{d_{out} \times d_{in}}$ and an activation matrix $X \in \mathbb{R}^{d_{in} \times N_{samples}}$ produced by passing the calibration dataset through preceding layers, the goal is to find a quantized matrix $\hat{W}$ that minimizes the squared Frobenius norm of output activation deviations:

$$\arg\min_{\hat{W}} \| W X - \hat{W} X \|_2^2 = \arg\min_{\hat{W}} \sum_{i=1}^{d_{\text{out}}} \| W_{i,:} X - \hat{W}_{i,:} X \|_2^2$$

Because the objective is independent across output channels (rows), we can isolate and solve the optimization problem for each row vector $w \in \mathbb{R}^{d_{in}}$:

$$\arg\min_{\hat{w}} (w - \hat{w}) X X^T (w - \hat{w})^T = \arg\min_{\hat{w}} \frac{1}{2} (w - \hat{w}) H (w - \hat{w})^T$$

where $H = 2 X X^T$ is the **Hessian matrix** of the layer's quadratic objective.

2. Derivation from Optimal Brain Surgeon (OBS)

Consider quantizing a single weight parameter w_q to its nearest discrete value $\hat{w}_q$, producing an error $\delta w_q = w_q - \hat{w}_q$. Using a second-order Taylor expansion around the unquantized weights:

$$\Delta E = \delta w^T \cdot \nabla_w E + \frac{1}{2} \delta w^T H \delta w$$

Because the model has converged to a local minimum during pre-training, the gradient vector is near zero ($\nabla_w E \approx 0$), simplifying the error formulation to:

$$\Delta E \approx \frac{1}{2} \delta w^T H \delta w$$

We must find an update vector δw that compensates for quantizing index q, subject to the linear constraint that the q-th element changes by exactly δw_q:

$$\min_{\delta w} \frac{1}{2} \delta w^T H \delta w \quad \text{subject to} \quad e_q^T \delta w = \delta w_q$$

where e_q is the elementary basis vector with 1 at index q and 0 elsewhere.

Constructing the Lagrangian:

$$L(\delta w, \lambda) = \frac{1}{2}\, \delta w^T H\, \delta w + \lambda\, (e_q{}^T\, \delta w - \delta w_q)$$

Taking the partial derivative with respect to δw and setting it to zero:

$$\frac{\partial L}{\partial \delta w} = H\, \delta w + \lambda\, e_q = 0 \Longrightarrow \delta w = -\lambda\, H^{-1}\, e_q$$

Applying the constraint $e_q{}^T\, \delta w = \delta w_q$:

$$e_q{}^T\, (-\lambda\, H^{-1}\, e_q) = \delta w_q \Longrightarrow -\lambda\, [H^{-1}]_{qq} = \delta w_q \Longrightarrow \lambda = -\frac{\delta w_q}{[H^{-1}]_{qq}}$$

Substituting λ back into the update equation yields the **Optimal Brain Surgeon Parameter Compensation Formula**:

$$\delta w = \frac{\delta w_q}{[H^{-1}]_{qq}}\, H^{-1}\, e_q, \quad \mathbf{w}_{\text{updated}} = \mathbf{w} - \frac{w_q - \text{quant}(w_q)}{[H^{-1}]_{qq}} \cdot (H^{-1})_{:,\, q}$$

Conceptual Meaning:

When parameter w_q is rounded to a quantized grid point, **every other unquantized parameter in that layer is updated proportionally to the q-th column of the inverse Hessian**. This shifts surrounding weights to compensate for the quantization error introduced by w_q.

3. Algorithmic Innovations in GPTQ

Directly applying the classic OBS formula to an entire LLM would take months of compute because computing and updating H^{-1} for every single scalar parameter scales at $O(d_{in}{}^3)$. GPTQ introduces three algorithmic optimizations to make this tractable:

1. **Cholesky Pre-Computation:** The classic OBS algorithm requires updating the inverse Hessian matrix via rank-1 Gaussian elimination at each scalar step. GPTQ demonstrates that the sequence of scale denominators can be precomputed in a single pass using the Cholesky decomposition of the inverse Hessian:

   $$H^{-1} = L \cdot L^T$$

   where L is a lower triangular matrix. This stabilizes the computation and avoids re-inverting matrices mid-loop.

2. **Lazy Batch Updates (Block-Wise Quantization):** Rather than updating the entire remaining weight matrix after every single parameter, GPTQ quantizes parameters in **blocks of columns** (e.g., $B = 128$ columns at a time). The errors are accumulated locally inside fast on-chip GPU SRAM, and the large remaining weight matrix is updated using high-performance matrix-matrix multiplications (GEMM):

$$W_{:,\,\text{remain}} \leftarrow W_{:,\,\text{remain}} - \Delta W_{\text{block}} \cdot \left([H^{-1}{}_{\text{block}}]^{-1} \cdot H^{-1}{}_{\text{block, remain}} \right)$$

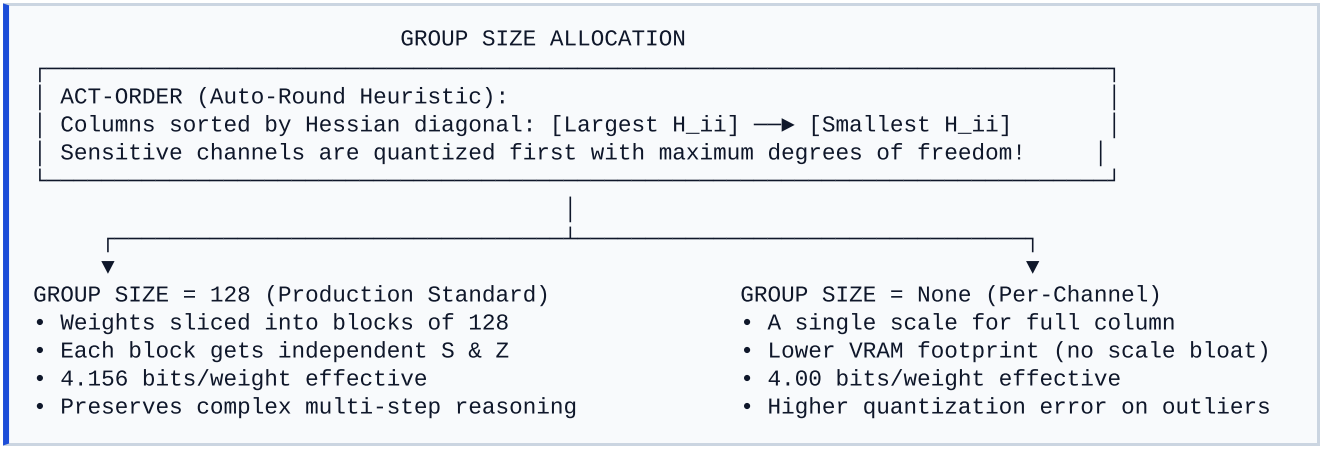
3. **Hessian Diagonal Damping:** When activations exhibit collinearity, the empirical matrix $X X^T$ can become ill-conditioned or non-invertible. GPTQ adds a numerical ridge dampening term to the diagonal before inverting:

$$\hat{H} = 2\, X X^T + \lambda_{\text{damp}} \cdot I$$

where $\lambda_{damp} = 0.01 \times \text{mean}(\text{diag}(H))$.

11.6 GPTQ Calibration Process and Group-Size Trade-Offs

The primary hyperparameter governing GPTQ efficiency and fidelity is the **Group Size (g)**.



1. Act-Order (Activation Ordering / Descriptive Sorting)

By default, GPTQ quantizes columns in their natural structural order (from column 1 to column d_{in}). However, some activation channels carry significantly larger variance than others.

- **The Act-Order Heuristic:** Sorts the columns of the weight matrix by their corresponding diagonal elements in the Hessian matrix ($H_{ii} = 2 \sum x_i^2$) in descending order prior to quantization.
- **Why it Works:** Columns with the largest activation energy are quantized first. The remaining unquantized columns retain maximum parameter freedom to absorb and cancel out the quantization error.
- **Downside:** Act-Order shuffles the physical weight column layout, breaking uniform contiguous memory access and slowing down inference decoding kernels by up to 15%–20% unless using optimized modern runtimes like ExLlamaV2.

2. Group Size (g) Analysis

$$\text{Effective Bits Per Weight} = b + \frac{\text{16 bits (Scale)} + \text{4 bits (Zero-Point)}}{g}$$

For 4-bit quantization ($b = 4$):

- **Group Size 128:**

$$\text{Effective Bits} = 4 + \frac{16 + 4}{128} = 4 + 0.156 = \mathbf{4.156\ bits/weight}$$

- **Group Size 32:**

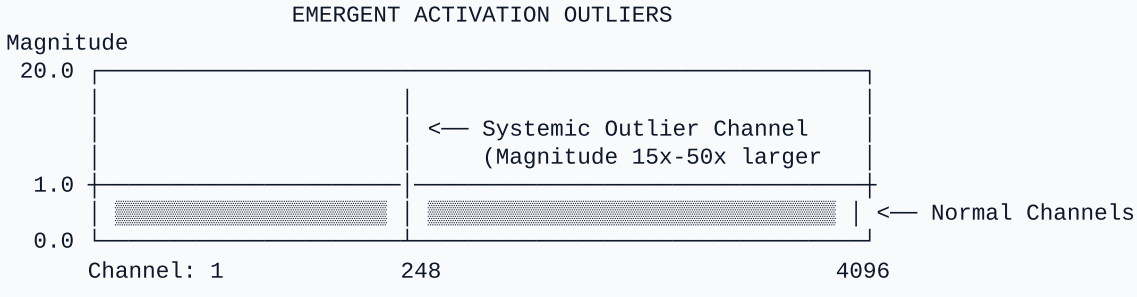
$$\text{Effective Bits} = 4 + \frac{16 + 4}{32} = 4 + 0.625 = \mathbf{4.625\ bits/weight}$$

Group Size Decision Guidelines:

- ***g* = *128* (Production Baseline):** Provides the optimal balance between compression ratio and model accuracy. It isolates outlier weights to blocks of 128 parameters without bloating scale metadata memory.
- ***g* = *64* or *32*:** Essential when quantizing smaller models (≤ *3*B) or when targeting 3-bit quantization, where quantization noise is high.
- ***g* = None (Per-Channel / No Groups):** Slashes memory overhead by using only one scale factor per column. Viable on large models (*70*B+ parameters) where parameter redundancy absorbs quantization noise, but can cause degradation on 7B/8B models.

11.7 When GPTQ Underperforms: Outlier Weights and Their Impact

Despite the second-order error compensation of GPTQ, aggressive 4-bit quantization can occasionally cause performance regressions. These failures trace back to **emergent activation outliers**.



1. Emergent Activation Outliers (Dettmers et al., 2022)

As Transformer models scale beyond **6.7 Billion parameters**, the internal representations undergo a phase change:

- A tiny fraction of activation channels (less than **0.1%** of all dimensions) abruptly expand in magnitude, reaching values **15× to 50× larger** than normal coordinates.
- **Channel Invariance:** These outlier activations are not random noise; they concentrate in specific channels across tokens (e.g., channel 248 consistently across sequences) and track critical semantic context tokens (such as punctuation, syntax delimiters, and sentence boundaries).

2. The GPTQ Outlier Failure Mechanism

1. **Scale Saturation:** When a weight interacts with an outlier activation channel, any slight error in that weight parameter is multiplied by an order of magnitude:

   $$\text{Error Contribution} = (w_q - \hat{w}_q) \cdot X_{\text{outlier}}$$

2. **Hessian Inversion Instabilities:** Outlier channels cause the condition number of the Hessian matrix ($\kappa(H) = \lambda_{max} / \lambda_{min}$) to spike into ill-conditioned territory ($> 10^8$). Even with damping, inverting an ill-conditioned matrix introduces numerical errors into Cholesky updates.
3. **The AWQ Alternative: AWQ (Activation-Aware Weight Quantization)** addresses this vulnerability. Instead of using second-order Hessian error compensation across all weights, AWQ identifies the top 1% of channels carrying outlier activations and **protects them from quantization entirely**, or calculates per-channel scale transforms to balance magnitude between activations and weights prior to RTN rounding.

11.8 Practical GPTQ Quantization Workflow with AutoGPTQ

Below is a complete, runnable production script using the `auto_gptq` library and Hugging Face `transformers` to quantize a continuous FP16/BF16 checkpoint into an optimized 4-bit GPTQ model.

```
import os
import torch
from transformers import AutoTokenizer
from auto_gptq import AutoGPTQForCausalLM, BaseQuantizeConfig

# ================================================================================
# 1. SETUP MODEL PATHS & CONFIGURATION
# ================================================================================

model_id = "meta-llama/Meta-Llama-3-8B-Instruct"
quantized_output_dir = "./Meta-Llama-3-8B-Instruct-GPTQ-4bit"

# Initialize Tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_id, use_fast=True)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

# Define GPTQ Quantization Architecture Specification
quantize_config = BaseQuantizeConfig(
    bits=4,                                 # Target 4-bit precision
    group_size=128,                         # Standard production group size
    desc_act=True,                          # Enable Act-Order (Activation Ordering)
    damp_percent=0.01,                      # Damping factor for Hessian stability (1%)
    sym=True,                               # Symmetric quantization (Z = 0)
    true_sequential=True,                   # Quantize layers sequentially, matching
inference flow
    model_seqlen=2048                       # Calibration context window length
)

# ================================================================================
# 2. PREPARE CALIBRATION DATASET
# ================================================================================

def build_production_calibration_data(tokenizer, num_samples: int = 128, seq_len: int =
2048):
    """
    Constructs a balanced calibration dataset capturing natural language,
    structured JSON, and code syntax.
    """
    raw_calibration_corpus = [
        "def quicksort(arr): return arr if len(arr) <= 1 else quicksort([x for x in arr[1:]
if x < arr[0]]) + [arr[0]] + quicksort([x for x in arr[1:] if x >= arr[0]])",
        "The Gross Domestic Product (GDP) expanded at an annualized rate of 2.8% in the
third quarter, reflecting resilient consumer spending.",
        "System Policy: All database connections must authenticate using TLS 1.3 mutual
certificates. Cleartext traffic on port 5432 is strictly prohibited.",
        "JSON Response: {\"status\": \"success\", \"code\": 200, \"data\": {\"user_id\":
9812, \"verified\": true, \"roles\": [\"admin\", \"ops\"]}}"
    ]

    # Expand to fill desired sample count
    full_corpus = (raw_calibration_corpus * ((num_samples // len(raw_calibration_corpus)) +
1))[:num_samples]

    calibration_dataset = []
    for text in full_corpus:
        # Tokenize and format as expected by AutoGPTQ
        tokens = tokenizer(
            text,
            truncation=True,
            max_length=seq_len,
            padding="max_length",
            return_tensors="pt"
        )
        calibration_dataset.append({
            "input_ids": tokens["input_ids"].squeeze(0),
```

```
            "attention_mask": tokens["attention_mask"].squeeze(0)
        })

    return calibration_dataset

print("Constructing calibration data...")
calibration_data = build_production_calibration_data(tokenizer, num_samples=128,
seq_len=2048)


# ================================================================================
# 3. INSTANTIATE UNQUANTIZED MODEL & EXECUTE GPTQ
# ================================================================================

print("Loading FP16 base model onto host system...")
model = AutoGPTQForCausalLM.from_pretrained(
    model_id,
    quantize_config=quantize_config,
    low_cpu_mem_usage=True,
    torch_dtype=torch.float16,
    device_map="auto"
)

print("Starting GPTQ Hessian Calibration and Weight Quantization...")
# This step computes Hessian matrices via layer-wise forward passes
# and updates weights block-by-block using the Cholesky OBS formulation.
model.quantize(
    calibration_data,
    batch_size=1,
    cache_examples_on_gpu=False               # Offload activations to save VRAM on single
GPU
)

# ================================================================================
# 4. SAVE COMPRESSED MODEL ARTIFACTS
# ================================================================================

print(f"Quantization complete. Saving 4-bit artifacts to {quantized_output_dir}...")
os.makedirs(quantized_output_dir, exist_ok=True)
model.save_quantized(quantized_output_dir, use_safetensors=True)
tokenizer.save_pretrained(quantized_output_dir)

print("Artifacts successfully generated:")
for filename in os.listdir(quantized_output_dir):
    print(f" - {filename}")
```

Technical Summary of Module 11 Part 1

Section	Algorithmic / Core Mechanism	Mathematical Formulation / Invariant	Primary Failure Mode Addressed
11.1 Quantization Basics	Scale (S) & Zero-Point (Z) mapping	$q = \text{clip}(\lfloor x/S \rceil + Z, q_{min}, q_{max})$; Symmetric: $Z = 0$	Integer underflow; hardware GEMM inefficiencies from non-zero offsets

Section	Algorithmic / Core Mechanism	Mathematical Formulation / Invariant	Primary Failure Mode Addressed
11.2 PTQ vs. QAT	Straight-Through Estimator (STE)	$\partial L / \partial w \approx (\partial L / \partial \hat{w}) \cdot \mathbb{I}(q_{min} \le w / S \le q_{max})$	Vanishing gradients during discrete rounding backpropagation
11.3 Calibration Data	Hessian curvature modeling ($H = 2XX^T$)	$\kappa(H) = \lambda_{max} / \lambda_{min}$; Domain coverage	Domain divergence; RoPE failure on truncated sequence lengths
11.4 Hardware Trade-Offs	Memory-bound autoregressive decoding	$\text{Tokens/sec} \approx \text{Bandwidth (GB/s)} / \text{Model Footprint (GB)}$	Memory bandwidth saturation bottlenecking single-stream token generation
11.5 GPTQ Algorithm	Optimal Brain Surgeon (OBS) second-order update	$\delta w = -((w_q - \text{quant}(w_q)) / [H^{-1}]_{qq}) \cdot (H^{-1})_{:,q}$	Error accumulation during naive scalar rounding
11.6 Group Sizes & Act-Order	Sub-channel grouping & activation sorting	$\text{Bits} = 4 + 20 / g$; Sort columns descending by H_{ii}	Global scale inflation caused by isolated channel outliers
11.7 Outlier Dynamics	Emergent activation outliers (> 6.7B)	Massive values in $< 0.1\%$ of activation coordinates	Outlier-induced scale collapse and ill-conditioned Hessian inversions
11.8 AutoGPTQ Workflow	Production Python quantization pipeline	`BaseQuantizeConfig(bits=4, group_size=128)`	Fragmented, error-prone manual quantization scripts